Lecture Notes: TwinCAT 3

TwinCAT Automation Interface

March 14, 2026

Contents

1 Introduction to the Automation Interface **2**
- 1.1 Manual Tasks that can be Automated . . . 2
- 1.2 Real-World Example . . . 2

2 System Architecture and Requirements **2**
- 2.1 Supported Languages . . . 2
- 2.2 Platform Constraint . . . 2
- 2.3 Installation . . . 3

3 Setting Up the Development Environment (C#) **3**
- 3.1 Required References . . . 3

4 The ITcSysManager "God Class" **5**
- 4.1 Version Management . . . 5
- 4.2 Key Methods . . . 5

5 Troubleshooting: The COM Message Filter **8**
- 5.1 The Fix . . . 8
- 5.2 UI Management . . . 8

6 Deployment Workflow **11**

Quick Reference Card **15**

1 Introduction to the Automation Interface

The **TwinCAT Automation Interface** is used to "automate the automation" by enabling the programmatic creation and manipulation of TwinCAT XAE configurations through scripting or programming code. In large projects involving multiple PLCs, it saves significant time and increases quality by removing the need for manual repetition of setup tasks.

1.1 Manual Tasks that can be Automated

- Defining real-time properties and adding/removing tasks.
- Creating Program Organization Units (POUs) and writing business logic.
- Installing and referencing TwinCAT system or user libraries.
- Defining inputs/outputs and linking them to POU instances.
- Configuring targets, setting IP addresses, and creating AMS routes.
- Activating configurations and running static code analysis.

1.2 Real-World Example

The **ESO Extremely Large Telescope** project used this interface to automatically configure and update software for 132 separate Beckhoff PLCs.

Task	Manual Time	Automated Time
Configure 1 PLC	30–60 minutes	2–5 minutes
Configure 132 PLCs	66–132 hours	4–11 hours
Update all PLCs	Hours per change	Minutes total

Table 1: Time Savings with Automation Interface

2 System Architecture and Requirements

To fully automate TwinCAT tasks, two primary components are required:

1. **Visual Studio DTE (Development Tools Environment):** Provides programmatic access to standard Visual Studio functionality (find/replace, build management, error lists) upon which TcXaeShell is based.
2. **TwinCAT Automation Interface:** Provides access to Beckhoff-specific additions, such as PLC project handling, variable linking, and I/O management.

2.1 Supported Languages

Bindings exist for **C++** and **C#**, as well as scripting languages like **PowerShell** and **IronPython**. C# is often preferred due to the extensive documentation available.

2.2 Platform Constraint

While the automation code can be written in cross-platform frameworks like .NET 6, it will **only run on Windows** because the TwinCAT XAE environment itself is Windows-based.

2.3 Installation

The interface is automatically installed whenever the TwinCAT XAE is installed on a machine.

> **Note:** The Automation Interface requires TwinCAT XAE (the engineering environment) to be installed—not just the XAR runtime. This is because the interface controls the Visual Studio-based development environment.

3 Setting Up the Development Environment (C#)

To create an automation program, you must add specific references to your project:

- **COM Reference:** Select the latest version of the "TwinCAT Automation Interface".
- **NuGet Packages:** Install `EnvDTE` and `EnvDTE80` for Visual Studio functionality.
- **Command Line Parsing:** For professional console applications, a library like `NDesk.Options` is recommended to handle parameters such as the Visual Studio solution path and the target AmsNetId.

3.1 Required References

Reference Type	Name	Purpose
COM	Beckhoff TwinCAT XAE	TwinCAT-specific automation
NuGet	EnvDTE	VS automation core
NuGet	EnvDTE80	VS automation extensions
NuGet	NDesk.Options	Command line parsing

Table 2: Required Project References

Example: Command Line Parsing

```
// Command Line Parsing with NDesk.Options
// This allows the automation tool to accept parameters from CI/CD
    pipelines

using System;
using System.Collections.Generic;
using NDesk.Options;

namespace TwinCATAutomation
{
    class Program
    {
        // Variables to store parsed command line arguments
        private static string solutionPath = string.Empty;
        private static string targetAmsNetId = string.Empty;
        private static bool showHelp = false;
        private static bool activateConfig = false;
        private static bool startRuntime = false;

        static void Main(string[] args)
```

```
        {
            // Initialize the OptionSet for parsing command line arguments
            // Each option maps a flag to a variable or action
            OptionSet options = new OptionSet()
            {
                // -s or --solution: Path to the TwinCAT solution file
                {
                    "s|solution=",
                    "Path to the TwinCAT solution file (.sln)",
                    v => solutionPath = v
                },

                // -a or --amsnetid: Target PLC's AmsNetId
                {
                    "a|amsnetid=",
                    "Target AmsNetId (e.g., 192.168.1.100.1.1)",
                    v => targetAmsNetId = v
                },

                // -c or --activate: Flag to activate configuration
                {
                    "c|activate",
                    "Activate configuration after opening",
                    v => activateConfig = (v != null)
                },

                // -r or --run: Flag to start TwinCAT runtime
                {
                    "r|run",
                    "Start TwinCAT runtime after activation",
                    v => startRuntime = (v != null)
                },

                // -h or --help: Display help information
                {
                    "h|help",
                    "Show this help message",
                    v => showHelp = (v != null)
                }
            };

            // Parse the command line arguments
            List<string> extraArgs;
            try
            {
                extraArgs = options.Parse(args);
            }
            catch (OptionException ex)
            {
                Console.WriteLine($"Error: {ex.Message}");
                Console.WriteLine("Use --help for usage information.");
                return;
            }

```

```
            // Display help if requested or if required parameters are missing
            if (showHelp || string.IsNullOrEmpty(solutionPath))
            {
                Console.WriteLine("TwinCAT Automation Tool");
                Console.WriteLine("Usage: TwinCATAutomation.exe [OPTIONS]");
                Console.WriteLine();
                Console.WriteLine("Options:");
                options.WriteOptionDescriptions(Console.Out);
                return;
            }

            // Display parsed values
            Console.WriteLine($"Solution Path: {solutionPath}");
            Console.WriteLine($"Target AmsNetId: {targetAmsNetId}");
            Console.WriteLine($"Activate Config: {activateConfig}");
            Console.WriteLine($"Start Runtime: {startRuntime}");

            // Continue with automation logic...
            // RunAutomation(solutionPath, targetAmsNetId, activateConfig, startRuntime);
        }
    }
}

// Example command line usage:
// TwinCATAutomation.exe -s "C:\Projects\MyMachine.sln" -a "192.168.1.100.1.1" -c -r
// TwinCATAutomation.exe --solution="C:\Projects\MyMachine.sln" --amsnetid="127.0.0.1.1.1"
```

Tip: Using command line arguments makes your automation tool suitable for CI/CD pipelines like Jenkins, Azure DevOps, or GitHub Actions. The tool can be called with different parameters for different target PLCs without code changes.

4 The ITcSysManager "God Class"

The `ITcSysManager` is the main interface for the TwinCAT Automation Interface and is often referred to as a "god class" because it controls almost everything in the project.

4.1 Version Management

Beckhoff increments the interface number whenever new features are added (e.g., `ITcSysManager10`, `ITcSysManager15`). It is recommended to cast your project handle to the latest available version to access all features.

4.2 Key Methods

- `SetTargetNetId()`: Changes the target PLC for the currently open configuration.
- `ActivateConfiguration()`: Compiles the code and transfers it to the PLC.
- `StartRestartTwinCAT()`: Restarts the TwinCAT runtime to load the new software.

Interface	TwinCAT Version	Key Features Added
ITcSysManager	3.0	Basic automation
ITcSysManager10	3.1.4020	Enhanced PLC support
ITcSysManager13	3.1.4022	Static analysis
ITcSysManager15	3.1.4024	Latest features

Table 3: ITcSysManager Version History

Example: Initializing the DTE and SysManager

```
// Initializing Visual Studio DTE and TwinCAT SysManager
// This is the foundation for all automation operations

using System;
using System.IO;
using System.Runtime.InteropServices;
using EnvDTE;
using EnvDTE80;
using TCatSysManagerLib;  // TwinCAT Automation Interface COM reference

namespace TwinCATAutomation
{
    class AutomationEngine
    {
        private DTE2 dte;
        private Solution solution;
        private Project project;
        private ITcSysManager15 sysManager;

        public void Initialize(string solutionPath)
        {
            // Step 1: Create a DTE (Development Tools Environment) instance
            // The ProgID specifies which version of TcXaeShell to use
            // "TcXaeShell.DTE.15.0" corresponds to TwinCAT 3.1 with VS 2017/2019 shell

            Type dteType = Type.GetTypeFromProgID("TcXaeShell.DTE.15.0");

            if (dteType == null)
            {
                throw new Exception("TwinCAT XAE Shell not found. Is TwinCAT installed?");
            }

            // Create an instance of the DTE
            // This launches TcXaeShell in the background
            object dteObject = Activator.CreateInstance(dteType);
            dte = (DTE2)dteObject;

            Console.WriteLine("TcXaeShell instance created successfully");

```

```
            // Step 2: Configure DTE for background/automated operation
            // SuppressUI prevents dialog boxes from appearing
            // Visible = false runs the IDE in the background
            dte.SuppressUI = true;
            dte.MainWindow.Visible = false;

            // Alternative: Set Visible = true for debugging
            // dte.MainWindow.Visible = true;

            // Step 3: Open the TwinCAT solution file
            if (!File.Exists(solutionPath))
            {
                throw new FileNotFoundException($"Solution not found: {solutionPath}");
            }

            solution = dte.Solution;
            solution.Open(solutionPath);

            Console.WriteLine($"Solution opened: {solutionPath}");

            // Step 4: Access the first project in the solution
            // In TwinCAT, the solution typically contains one project
            // Index is 1-based in COM collections
            project = solution.Projects.Item(1);

            Console.WriteLine($"Project loaded: {project.Name}");

            // Step 5: Get the ITcSysManager interface from the project
            // Cast to the latest version (ITcSysManager15) for full functionality
            // The project.Object property returns the TwinCAT-specific interface
            sysManager = (ITcSysManager15)project.Object;

            Console.WriteLine("ITcSysManager15 interface acquired successfully");

            // The sysManager is now ready for automation operations
            // You can now:
            // - Set target NetId
            // - Activate configuration
            // - Start/stop runtime
            // - Access PLC projects, I/O tree, etc.
        }

        public void Cleanup()
        {
            // Properly close and release COM objects
            if (solution != null)
            {
                solution.Close(false);  // false = don't save changes
            }

            if (dte != null)
```

```
            {
                dte.Quit();
                Marshal.ReleaseComObject(dte);
            }

            Console.WriteLine("DTE instance closed and released");
        }
    }
}
```

```
// Alternative ProgIDs for different TwinCAT/Visual Studio versions:
//
// "TcXaeShell.DTE.15.0"     - TwinCAT XAE Shell (standalone, recommended)
// "VisualStudio.DTE.15.0"   - Visual Studio 2017 with TwinCAT integration
// "VisualStudio.DTE.16.0"   - Visual Studio 2019 with TwinCAT integration
// "VisualStudio.DTE.17.0"   - Visual Studio 2022 with TwinCAT integration
//
// Using TcXaeShell is recommended as it doesn't require a full VS installation
```

Warning: Always ensure proper cleanup of COM objects using `Marshal.ReleaseComObject()` and calling `dte.Quit()`. Failure to do so can leave orphaned TcXaeShell processes running in the background, consuming system resources.

5 Troubleshooting: The COM Message Filter

A common issue when using the Automation Interface is receiving a **COM Exception** stating "The message filter indicated that the application is busy". This occurs because the interface is communicating with Visual Studio via COM.

5.1 The Fix

You must implement a `MessageFilter` class (available in the Beckhoff documentation) that handles retries when the application is busy.

5.2 UI Management

For automated background tasks (like CI/CD pipelines), set the DTE properties `SuppressUI = true` and `Visible = false`.

Example: Registering the Message Filter

```
// Message Filter Implementation for COM Busy Handling
// This prevents "application is busy" exceptions during automation

using System;
using System.Runtime.InteropServices;

namespace TwinCATAutomation
```

```
{
    // COM Message Filter interface definition
    [ComImport]
    [Guid("00000016-0000-0000-C000-000000000046")]
    [InterfaceType(ComInterfaceType.InterfaceIsIUnknown)]
    public interface IOleMessageFilter
    {
        [PreserveSig]
        int HandleInComingCall(
            int dwCallType,
            IntPtr hTaskCaller,
            int dwTickCount,
            IntPtr lpInterfaceInfo);

        [PreserveSig]
        int RetryRejectedCall(
            IntPtr hTaskCallee,
            int dwTickCount,
            int dwRejectType);

        [PreserveSig]
        int MessagePending(
            IntPtr hTaskCallee,
            int dwTickCount,
            int dwPendingType);
    }

    // Message Filter implementation class
    public class MessageFilter : IOleMessageFilter
    {
        // Register the message filter with COM
        public static void Register()
        {
            IOleMessageFilter newFilter = new MessageFilter();
            IOleMessageFilter oldFilter = null;

            // Register our filter, get the old one (if any)
            int hr = CoRegisterMessageFilter(newFilter, out oldFilter
    );

            if (hr != 0)
            {
                Marshal.ThrowExceptionForHR(hr);
            }

            Console.WriteLine("COM Message Filter registered");
        }

        // Revoke (unregister) the message filter
        public static void Revoke()
        {
            IOleMessageFilter oldFilter = null;

            // Pass null to unregister our filter
            int hr = CoRegisterMessageFilter(null, out oldFilter);

            Console.WriteLine("COM Message Filter revoked");
        }
```

```

        // Handle incoming calls - always allow them
        int IOleMessageFilter.HandleInComingCall(
            int dwCallType,
            IntPtr hTaskCaller,
            int dwTickCount,
            IntPtr lpInterfaceInfo)
        {
            return 0;  // SERVERCALL_ISHANDLED
        }

        // Handle rejected calls - retry after a short delay
        int IOleMessageFilter.RetryRejectedCall(
            IntPtr hTaskCallee,
            int dwTickCount,
            int dwRejectType)
        {
            if (dwRejectType == 2)  // SERVERCALL_RETRYLATER
            {
                // Retry after 100ms
                return 100;
            }

            // Don't retry
            return -1;
        }

        // Handle pending messages - wait and don't dispatch
        int IOleMessageFilter.MessagePending(
            IntPtr hTaskCallee,
            int dwTickCount,
            int dwPendingType)
        {
            return 2;  // PENDINGMSG_WAITDEFPROCESS
        }

        // P/Invoke declaration for COM message filter registration
        [DllImport("ole32.dll")]
        private static extern int CoRegisterMessageFilter(
            IOleMessageFilter newFilter,
            out IOleMessageFilter oldFilter);
    }
}
```

```
// Using the Message Filter in Your Main Program
// Register at start, Revoke at end

using System;

namespace TwinCATAutomation
{
    class Program
    {
        static void Main(string[] args)
        {
            // CRITICAL: Register the message filter BEFORE any COM operations
            // This must be the first thing you do
```

```
            MessageFilter.Register();

            try
            {
                Console.WriteLine("Starting TwinCAT Automation...");

                // Create and initialize the automation engine
                AutomationEngine engine = new AutomationEngine();

                // Parse command line arguments (from previous example)
                string solutionPath = args.Length > 0 ? args[0] : "";
                string targetAmsNetId = args.Length > 1 ? args[1] : "127.0.0.1.1.1";

                // Initialize DTE and open solution
                engine.Initialize(solutionPath);

                // Perform automation tasks...
                // engine.SetTarget(targetAmsNetId);
                // engine.ActivateAndRun();

                // Cleanup
                engine.Cleanup();

                Console.WriteLine("Automation completed successfully");
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Error: {ex.Message}");
                Console.WriteLine(ex.StackTrace);
            }
            finally
            {
                // CRITICAL: Always revoke the message filter when done
                // This should be in a finally block to ensure it runs
                MessageFilter.Revoke();
            }
        }
    }
}
```

Tip: The Message Filter is essential for reliable automation. Without it, long-running operations like compilation or activation may randomly fail with "application is busy" errors, especially on slower machines or when the system is under load.

6 Deployment Workflow

The program should follow the same logical sequence as a manual user:

1. **Initialize DTE** and open the target solution.

2. **Set Target:** Use `SetTargetNetId` to point to the specific PLC (e.g., a local virtual machine at `127.0.0.1.1.1` or a remote CX-PLC).

3. **Deploy:** Call `ActivateConfiguration` and `StartRestartTwinCAT` to put the PLC into Run Mode.

Step 1	Step 2	Step 3	Step 4
Open Solution ↓ DTE.Solution.Open()	Set Target ↓ SetTargetNetId()	Activate ↓ ActivateConfiguration()	Start Runtime ↓ StartRestartTwinCAT()

Figure 1: Deployment Workflow Sequence

Example: Deployment Sequence

```
// Complete Deployment Sequence
// Sets target, activates configuration, and starts the TwinCAT runtime

using System;
using TCatSysManagerLib;

namespace TwinCATAutomation
{
    class DeploymentManager
    {
        private ITcSysManager15 sysManager;

        public DeploymentManager(ITcSysManager15 manager)
        {
            this.sysManager = manager;
        }

        /// <summary>
        /// Deploys the current configuration to the specified target PLC
        /// </summary>
        /// <param name="targetAmsNetId">Target PLC's AmsNetId (e.g., "192.168.1.100.1.1")</param>
        /// <param name="startRuntime">Whether to start the runtime after activation</param>
        public void Deploy(string targetAmsNetId, bool startRuntime = true)
        {
            Console.WriteLine("=== Starting Deployment ===");
            Console.WriteLine($"Target: {targetAmsNetId}");

            // Step 1: Set the target AmsNetId
            // This tells TwinCAT which PLC to deploy to
            // Common values:
            //   "127.0.0.1.1.1" - Local machine (development/testing)
            //   "192.168.x.x.1.1" - Remote PLC via network
            Console.WriteLine("Setting target AmsNetId...");
            sysManager.SetTargetNetId(targetAmsNetId);
```

```
            Console.WriteLine($"Target set to: {targetAmsNetId}");

            // Step 2: Activate the configuration
            // This compiles the PLC code and transfers it to the target
            // The process includes:
            //   - Code compilation and error checking
            //   - Configuration validation
            //   - Transfer to target PLC
            //   - License validation
            Console.WriteLine("Activating configuration...");

            try
            {
                // ActivateConfiguration parameters:
                // - None in basic form
                // - Returns when activation is complete
                sysManager.ActivateConfiguration();
                Console.WriteLine("Configuration activated successfully");
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Activation failed: {ex.Message}");
                throw;  // Re-throw to stop deployment
            }

            // Step 3: Start or restart the TwinCAT runtime
            // This puts the PLC into Run mode
            if (startRuntime)
            {
                Console.WriteLine("Starting TwinCAT runtime...");

                try
                {
                    // StartRestartTwinCAT restarts the TwinCAT system service
                    // This loads and runs the newly activated configuration
                    // The PLC will transition from Config mode to Run mode
                    sysManager.StartRestartTwinCAT();
                    Console.WriteLine("TwinCAT runtime started - PLC is now in RUN mode");
                }
                catch (Exception ex)
                {
                    Console.WriteLine($"Failed to start runtime: {ex.Message}");
                    throw;
                }
            }
            else
            {
                Console.WriteLine("Runtime start skipped (configuration only)");
            }
```

```

            Console.WriteLine("=== Deployment Complete ===");
        }

        /// <summary>
        /// Deploys to multiple targets sequentially
        /// </summary>
        public void DeployToMultipleTargets(string[] targetNetIds)
        {
            Console.WriteLine($"Deploying to {targetNetIds.Length} targets...");

            foreach (string netId in targetNetIds)
            {
                try
                {
                    Deploy(netId, startRuntime: true);
                    Console.WriteLine($"[SUCCESS] {netId}");
                }
                catch (Exception ex)
                {
                    Console.WriteLine($"[FAILED] {netId}: {ex.Message}");
                    // Continue with next target or throw based on requirements
                }
            }
        }
    }
}
```

```
// Complete Main Program Example
// Combines all components into a working automation tool

using System;

namespace TwinCATAutomation
{
    class Program
    {
        static void Main(string[] args)
        {
            // Register COM message filter first
            MessageFilter.Register();

            AutomationEngine engine = null;

            try
            {
                // Parse arguments (simplified)
                string solutionPath = @"C:\Projects\MyMachine\MyMachine.sln";
                string targetAmsNetId = "127.0.0.1.1.1";  // Local for testing

                // Override from command line if provided
                if (args.Length >= 1) solutionPath = args[0];
                if (args.Length >= 2) targetAmsNetId = args[1];
```

```

                // Initialize
                engine = new AutomationEngine();
                engine.Initialize(solutionPath);

                // Get the SysManager from engine (you'd add this property)
                // ITcSysManager15 sysManager = engine.SysManager;

                // Deploy
                // DeploymentManager deployer = new DeploymentManager(sysManager);
                // deployer.Deploy(targetAmsNetId, startRuntime: true);

                Console.WriteLine("Deployment successful!");
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Deployment failed: {ex.Message}");
                Environment.ExitCode = 1;  // Non-zero exit code for CI/CD
            }
            finally
            {
                engine?.Cleanup();
                MessageFilter.Revoke();
            }
        }
    }
}
```

Warning: When deploying to production PLCs, always verify the target AmsNetId carefully. Deploying to the wrong PLC can cause unexpected machine behavior or downtime. Consider implementing a confirmation step or target validation in your automation scripts.

Quick Reference Card

Required References

Type	Reference
COM	Beckhoff TwinCAT XAE
NuGet	EnvDTE, EnvDTE80
NuGet	NDesk.Options (optional)

DTE ProgIDs

Key ITcSysManager Methods

ProgID	Description
TcXaeShell.DTE.15.0	TwinCAT XAE Shell (standalone)
VisualStudio.DTE.16.0	VS 2019 with TwinCAT
VisualStudio.DTE.17.0	VS 2022 with TwinCAT

```
// Set target PLC
sysManager.SetTargetNetId("192.168.1.100.1.1");

// Compile and transfer
sysManager.ActivateConfiguration();

// Start runtime
sysManager.StartRestartTwinCAT();
```

Essential Startup/Cleanup Pattern

```
MessageFilter.Register();     // First line
try {
    // Automation code here
} finally {
    MessageFilter.Revoke();   // Always revoke
}
```

Last Updated: March 14, 2026